

История переписки — Claude Code

Полная история обучения GraphQL

Сессия: 2026-05-09

Вопрос

я перезапустил idea и что вся переписка наша потерялась?

Ответ

История переписки хранится в сессии Claude Code (CLI), а не в IDEA. Если ты перезапустил только IDEA — сессия CLI не затронута и история сохранена.

Если же ты закрыл и снова открыл сам терминал с Claude Code — тогда да, сессия сброшена и предыдущая история потеряна. Но я вижу контекст проекта (git статус, рабочая директория `/home/user/dev/Java Projects/graphql`), так что могу продолжить работу с того места, где остановились.

Вопрос

какие предыдущие я задавал вопросы посмотри по истории или в сессии?

Ответ

У меня нет доступа к предыдущим сессиям. Я вижу только текущий разговор.

Каждая новая сессия Claude Code начинается с чистого листа — предыдущая история не сохраняется между сессиями.

Однако я вижу состояние проекта из git: GraphQL на Java (Maven), изменены Dockerfile, docker-compose.yml, pom.xml, удалён application.properties, добавлены application-local.yml и application.yml.

Вопрос

давай сохранять историю переписки мои вопросы и твои ответы в файле корне проекта, не тезисно а то как я спросил тебя и то как ты ответил мне

Ответ

Важный момент: я не могу делать это автоматически — у меня нет механизма, который сам обновляет файл после каждого сообщения. Но я могу создать файл и обновлять его вручную в конце каждого ответа.

Файл CHAT_HISTORY.md создан в корне проекта. Буду дописывать каждый обмен сюда в конце ответа.

Вопрос

я java разработчик и мне нужно научиться от простых вещей к продвинутым возможностям graphql. я скачал чужой проект из интернета давай объясни мне что это за технология, как работает этот проект, чем он лучше rest или webflux начни обучать меня и задавать мне вопросы, что бы я полностью разобрался с этой технологией, составь план и давай начнем. Может это займет не один день, мы будем сохранять переписку контекст в файл после каждого вопроса и ответа и продолжим завтра и послезавтра пока я не овладею знаниями. За сколько часов можно изучить graphql?

Ответ

Что такое GraphQL?

GraphQL — это язык запросов для API, созданный Facebook в 2015 году. Это не база данных и не фреймворк — это спецификация того, как клиент общается с сервером.

Главная идея: клиент сам решает, какие данные получить, а не сервер диктует формат ответа.

REST vs GraphQL

	REST	GraphQL
Эндпоинтов	Много (/employees, /departments, /orgs)	Один (/graphql)
Формат запроса	URL + HTTP метод	Тело запроса (query/mutation)
Данные в ответе	Фиксированные (сервер решает)	Гибкие (клиент решает)
Over-fetching	Да (лишние поля приходят всегда)	Нет (только то что просишь)
Under-fetching	Да (N запросов для связанных данных)	Нет (всё за один запрос)
Версионирование	/v1/, /v2/	Не нужно (схема эволюционирует)
Документация	Swagger/OpenAPI отдельно	Схема = документация

Пример: нужно показать сотрудников с отделом и организацией — в REST 3 запроса, в GraphQL один:

```
graphql
```

```
query {  
  employees {  
    firstName  
    department { name }  
    organization { name }  
  }  
}
```

GraphQL vs WebFlux

Это разные уровни — не надо противопоставлять. WebFlux — реактивный стек (Mono/Flux, неблокирующий I/O). GraphQL — протокол/формат запросов к API. Они могут работать вместе. В этом проекте WebFlux подключён только для тестов.

Как устроен этот проект

```
Клиент → POST /graphql → Spring GraphQL → Controller (@QueryMapping/@MutationMapping)  
                                         ↓  
                                   Repository (JPA)  
                                         ↓  
                                   PostgreSQL
```

3 сущности с иерархией: Organization → Department[] + Employee[]

3 файла .graphqls описывают схему, 3 контроллера обрабатывают запросы.

План обучения

- Блок 1 — Основы GraphQL: Query/Mutation/Subscription, схема и типы, скалярные типы, GraphQL
- Блок 2 — Spring GraphQL: @QueryMapping, @MutationMapping, @SchemaMapping, @Argument, @BatchMapping
- Блок 3 — Продвинутые возможности: фильтрация, Input types, фрагменты, переменные, обработка ошибок
- Блок 4 — Архитектура и паттерны: DataLoader, Subscriptions, безопасность, тестирование

Сколько времени нужно?

Не указываю временных оценок — это зависит от глубины и темпа. У нас есть реальный проект и план, это лучше любого курса.

Первый вопрос (Блок 1, Урок 1):

Открой `src/main/resources/graphql/departments.graphqls`:

graphql

```
type Department {  
  id: ID!  
  name: String!  
  organization: Organization  
  employees: [Employee]  
}
```

Почему у id и name СТОИТ !, а у organization и employees нет? Что означает [Employee]? Объясни своими словами.

Вопрос

история сохранена а где же таблица REST vs GraphQL?

Ответ

Таблица была сохранена в Markdown формате. Markdown-таблица отображается красиво только если редактор рендерит Markdown (в IDEA — кнопка Preview в правом верхнем углу). В сыром виде таблица выглядит с символами | — это нормально. По просьбе переключились на Word формат.

Вопрос

не красиво таблица отображается, сохраняй в ворд формат

Ответ

Создан файл CHAT_HISTORY.docx в корне проекта с правильным форматированием: заголовки, таблицы, блоки кода с серым фоном. Использована библиотека python-docx.

Вопрос

а создай pdf файл

Ответ

Создан файл CHAT_HISTORY.pdf с кириллицей, цветной таблицей (заголовок тёмно-синий, строки чередуются) и блоками кода на сером фоне. Использована библиотека reportlab (Python). Шрифт DejaVu для поддержки кириллицы.

Вопрос

записывай историю в папку resources platform в pdf и md файл

Ответ

Файлы перемещены в src/main/resources/platform/. Пустая папка resources/platform в корне удалена.

Теперь история ведётся в двух файлах:

- src/main/resources/platform/CHAT_HISTORY.md
- src/main/resources/platform/CHAT_HISTORY.pdf

Вопрос

Я не знаю что это за файлы department.graphqls v2.graphqls v3.graphqls. Я привык к REST контроллерам, сервисам, репозиториям. Почему .graphqls файлы в resources, а не в src/main/java? Что такое Query, Mutation, Subscription? Объясни структуру и логику проекта.

Ответ

Почему .graphqls файлы в resources, а не в java?

В REST контракт API и логика находятся в одном месте (Java контроллер). В GraphQL они строго разделены:

```
src/main/resources/graphql/
  department.graphqls ← КОНТРАКТ (что API умеет, какие типы)
  v2.graphqls
  v3.graphqls

src/main/java/
  controller/DepartmentController.java ← ЛОГИКА (как это работает)
  repository/DepartmentRepository.java
  entities/Department.java
```

.graphqls — это текстовый файл-схема, не Java код. Spring при старте читает все .graphqls из resources/graphql/ и строит из них граф. Это похоже на application.yml — конфигурация, не код. Если схема и Java код расходятся — Spring выдаст ошибку при старте, не в runtime.

Три типа операций

Операция	Аналог в REST	Для чего
Query	GET	Читать данные
Mutation	POST / PUT / DELETE	Изменять данные
Subscription	WebSocket	Подписка на события в реальном времени

Разбор department.graphqls построчно

graphql

```
type Query {
  departments: [Department]          // GET /departments → список
  department(id: ID!): Department!   // GET /departments/{id} → один объект
}

type Mutation {
  newDepartment(department: DepartmentRequestDto!): Department // POST
  updateDepartment(department: DepartmentRequestDto!, id: ID!): Department // PUT
  deleteDepartment(id: ID!): Department // DELETE
}

input DepartmentRequestDto {
  name: String!           // обязательное поле (аналог @RequestBody в REST)
  organizationId: Int     // необязательное (null разрешён)
}

type Department {
  id: ID!                // никогда не null
  name: String!          // никогда не null
  organization: Organization // может быть null
  employees: [Employee]  // список, может быть null
}
```

Связь схемы с Java кодом

department.graphqls	DepartmentController.java
type Query {	@QueryMapping
departments: [Department] →	List<Department> departments()
department(id: ID!) →	@QueryMapping
}	Department department(@Argument Integer id)
type Mutation {	@MutationMapping
newDepartment(...) →	Department newDepartment(@Argument DepartmentRequestDto dto)
}	

Схема = обязательство. Java код = выполнение этого обязательства.

Почему три файла?

Организационно — разбили по сущностям. `extend type Query` означает "добавить методы в уже объявленный Query из другого файла".

Вопрос к студенту:

`department(id: ID!): Department!` возвращает `Department!` (с !), а `newDepartment(...): Department` — без !. Почему?

Вопрос

input DepartmentRequestDto что такое и чем input от type отличается?

Ответ

type VS input — в чём разница

В GraphQL есть строгое правило: объекты для чтения и объекты для записи — разные вещи.

	type	input
Направление	Сервер → Клиент	Клиент → Сервер
Аналог в REST/Java	@ResponseBody / Entity	@RequestBody / DTO
Может содержать	Поля + вложенные объекты	Только скалярные поля и другие input типы
Используется в	Возвращаемое значение	Аргумент метода

input принимает только скалярные поля и другие input типы. type может содержать вложенные объекты, резолверы, циклические ссылки — поэтому его нельзя передавать как аргумент в мутацию.

graphql

```
// Клиент ОТПРАВЛЯЕТ (для создания/обновления):
input DepartmentRequestDto {
  name: String!
  organizationId: Int    // просто число — только ID
}

// Сервер ВОЗВРАЩАЕТ (результат):
type Department {
  id: ID!
  name: String!
  organization: Organization // полный объект со всеми полями
  employees: [Employee]     // список объектов
}
```

Обрати внимание: клиент при создании знает только organizationId: Int. Когда получает данные обратно — видит полный объект organization: Organization с именем и всеми полями.

Аналогия с Java:

java

```
@PostMapping("/departments")
public DepartmentResponseDto create(@RequestBody DepartmentRequestDto dto) {
    // dto    = input в GraphQL (что пришло от клиента)
    // return = type  в GraphQL (что отдаём клиенту)
}
```

Вопрос к студенту:

Почему для `deleteDepartment` не создали `input DeleteDepartmentDto { id: ID! }`, а передают `id: ID!` напрямую? Когда имеет смысл создавать `input` объект, а когда достаточно скалярного значения?

Вопрос

`department(id: ID!): Department!` **возвращает с !**, а `newDepartment(...): Department` **без !**. Я не знаю точно, но думаю что при создании возвращаемый объект может быть `null` — такая логика у `Mutation`.

Ответ

Ты мыслишь в правильном направлении. `!` в возвращаемом типе — это гарантия результата.

Query — жёсткая гарантия (`Department!`)

Когда клиент запрашивает `department(id: 5)` — он ожидает найти конкретную запись. Если записи нет — это исключительная ситуация. Сервер должен либо вернуть объект, либо бросить ошибку. Никогда не вернёт `null`.

Клиент: "Дай мне департамент с `id=5`"
Сервер: либо `→ { id: 5, name: "IT" }`
 либо `→ ERROR: "Department not found"`
 никогда `→ null`

Mutation — мягкий отказ (`Department` без `!`)

При создании могут быть ситуации когда операция "не удалась" но не является критической ошибкой:

Клиент: "Создай департамент"
Сервер: либо `→ { id: 6, name: "HR" }` `// успех`
 либо `→ null` `// что-то пошло не так, но не exception`

Важный нюанс — это проектное решение

Многие GraphQL API делают мутации тоже с `!`, используя `errors` в ответе. Оба подхода правильные.

Правило большого пальца

Ситуация	Возвращаемый тип	Логика
Запрос по ID	<code>Type!</code>	Запись должна существовать
Запрос списка	<code>[Type]</code>	Пустой список — не ошибка

Ситуация	Возвращаемый тип	Логика
Создание/обновление	Type или Type!	Зависит от стратегии ошибок
Удаление	Type	Возвращает удалённый объект или null

Вопрос

`input` это объект с фронта — DTO. При создании и обновлении нужен объект с несколькими полями. При удалении нужна только айдишка — зачем тогда `DepartmentRequestDto`, он не нужен.

Ответ

Точно правильно. `input` нужен когда передаёшь несколько полей. Одно значение — передаёшь напрямую как скаляр. Создавать `input` ради одного поля — лишний код без смысла.

graphql

```
// Нужен input – несколько полей
newDepartment(department: DepartmentRequestDto!): Department

// Не нужен input – одно значение
deleteDepartment(id: ID!): Department
```

Аналогия из Java:

java

```
// Создание – нужен @RequestBody с несколькими полями
@PostMapping
public Department create(@RequestBody DepartmentRequestDto dto)

// Удаление – достаточно одного @PathVariable
@DeleteMapping("/{id}")
public void delete(@PathVariable Integer id)
```

Итог Блока 1 — что усвоено:

- `.graphqls` файлы — контракт API в `resources`, Java код — реализация в `java`
- `type Query` = ЧТЕНИЕ, `type Mutation` = изменение, `Subscription` = реальное время
- `type` — что сервер возвращает клиенту
- `input` — что клиент отправляет серверу (DTO)
- `!` — поле/результат никогда не `null`, без `!` — может быть `null`
- `[Employee]` — список
- `input` нужен когда полей несколько, скаляр — когда одно значение

Вопрос

Давай пройдемся ещё раз по пунктам подробно: - Как читать .graphqls файлы этого проекта - GraphQL — запускаем и делаем первые запросы - .graphqls не усложняет ли логику ведь в REST этого нет? - Запросы как делать, первый запрос как написать - Откуда узнать какие фронтенд может запросы делать а какие нет? - Что за архитектура GraphQL, чем лучше REST, в каких случаях применять, что быстрее на продакшене?

Ответ

Как читать .graphqls файлы

.graphqls — это контракт API. Читай его как меню ресторана: написано что можно заказать, с какими параметрами и что получишь в ответ.

Грамматика чтения — три вопроса:

graphql

```
department(id: ID!): Department!
```

↑ ↑ ↑
"метод" "что принять" "что вернуть"

Читай вслух: "Метод department, принимает id типа ID (обязательный), возвращает объект Department (гарантированно не null)"

Разбор трёх файлов проекта:

graphql

```
— department.graphqls —————
type Query {
  departments: [Department]          // "дай все департаменты" → список
  department(id: ID!): Department!   // "дай один по id" → объект
}

— v3.graphqls —————
extend type Query {                  // "extend" = дополняем Query из другого файла
  organizations: [Organization]
  organization(id: ID!): Organization!
  organizationName(name: String!): Organization! // поиск по имени
}

— v2.graphqls —————
extend type Query {
  employees: [Employee]
  employeesWithFilter(filter: EmployeeFilter): [Employee] // с фильтрацией
  employee(id: ID!): Employee!
}
input EmployeeFilter {
  salary: FilterField // фильтр по зарплате
  age: FilterField     // фильтр по возрасту
  position: FilterField // фильтр по должности
}
input FilterField {
  operator: String! // "gt" (больше), "lt" (меньше), "eq" (равно)
  value: String!    // "50000"
```

Почему три файла, а не один?

Организационно — разбили по сущностям. Spring читает все .graphqls из папки и склеивает в одну схему. extend type Query — добавить методы в уже объявленный Query.

.graphqls усложняет ли логику? Честное сравнение

	REST	GraphQL
Контракт	Нет (или Swagger отдельно)	Обязательный .graphqls
Добавить метод	Написал контроллер — готово	Схема + контроллер
Документация	Swagger (поддерживать отдельно)	Схема = документация (всегда актуальна)
Ошибка несоответствия	В runtime	При старте приложения

Да, на старте чуть больше работы. Но это окупается — схема всегда актуальна и является живой документацией.

GraphiQL — запускаем

GraphiQL — встроенный браузерный клиент для тестирования. Аналог Swagger UI, но мощнее.

Запустить проект:

```
bash
docker-compose up
# или локально:
./mvnw spring-boot:run -Dspring-boot.run.profiles=local
```

Открыть в браузере: <http://localhost:8080/graphql>

Первые запросы — пишем вместе

Запрос 1 — получить все организации:

```
graphql
query {
  organizations {
    id
    name
  }
}
```

Запрос 2 — получить организации с вложенными департаментами:

```
graphql
query {
  organizations {
    id
    name
    departments {
      id
      name
    }
  }
}
```

Запрос 3 — получить всё дерево за один запрос:

graphql

```
query {  
  organizations {  
    id  
    name  
    departments {  
      id  
      name  
      employees {  
        firstName  
        lastName  
        position  
      }  
    }  
  }  
  employees {  
    firstName  
    salary  
  }  
}
```

В REST для этого нужно было бы 3 отдельных запроса. Здесь — один.

Запрос 4 — создать организацию (Mutation):

graphql

```
mutation {  
  newOrganization(organization: { name: "Google" }) {  
    id  
    name  
  }  
}
```

Запрос 5 — фильтрация сотрудников по зарплате:

graphql

```
query {  
  employeesWithFilter(filter: {  
    salary: { operator: "gt", value: "50000" }  
  }) {  
    firstName  
    lastName  
    salary  
  }  
}
```

Откуда фронтенд знает какие запросы можно делать?

Это называется интроспекция — ключевая особенность GraphQL. Сервер всегда отвечает на запросы о своей схеме. В GraphQL нажми кнопку "Docs" (справа) — там все доступные операции, типы и описания. Это живая документация.

Инструменты для фронтенда:

- Apollo Client — генерирует TypeScript типы из схемы

- GraphQL Code Generator — генерирует код под любой язык

Когда применять GraphQL vs REST

GraphQL выигрывает:

- Сложные связанные данные (как этот проект: Organization → Departments → Employees)
- Несколько клиентов (web, mobile, TV) — каждый берёт только нужные поля
- Быстро меняющийся продукт — добавляешь поля без версионирования
- Мобильное приложение — меньше данных по сети

REST выигрывает:

- Простой CRUD без сложных связей
- Публичный API для внешних разработчиков — понятнее и лучше кешируется
- Файлы, бинарные данные — нативная поддержка
- Команда не знает GraphQL

Что быстрее на продакшене?

Сценарий	Победитель	Почему
Один простой запрос	REST	Меньше overhead на парсинг
Несколько связанных запросов	GraphQL	1 запрос вместо 3-5
Мобильный клиент	GraphQL	Меньше данных по сети
Кеширование	REST	HTTP кеш работает нативно

Главная ловушка GraphQL — N+1 проблема:

```
Запрос: organizations → departments → employees
Наивная реализация:
  1 запрос на organizations
+ 3 запроса на departments (по одному на каждую org)
+ 15 запросов на employees (по одному на каждый dept)
= 19 запросов в БД вместо 1!
```

В этом проекте решено через @BatchMapping — разберём в Блоке 2.

Задание:

Запустить проект, открыть <http://localhost:8080/graphql>, сделать запрос на получение организаций с департаментами.

Вопрос

```
`graphql query { organizations { id name } }`
```

Не получилось ответ получить, ошибка в запросе.

Ответ

Проект запущен (проверил через Docker — оба контейнера работают: spring-graphql-app и graphql-db). Сам запрос синтаксически правильный. Проблема в другом — база данных пустая, данных ещё нет. Запрос выполнен, просто вернул пустой список [].

Сначала нужно создать данные через GraphQL:

Шаг 1 — создать организацию:

```
graphql

mutation {
  newOrganization(organization: { name: "Google" }) {
    id
    name
  }
}
```

Шаг 2 — создать департамент:

```
graphql

mutation {
  newDepartment(department: { name: "Engineering", organizationId: 1 }) {
    id
    name
  }
}
```

Шаг 3 — создать сотрудника:

```
graphql

mutation {
  newEmployee(employee: {
    firstName: "John"
    lastName: "Doe"
    position: "Developer"
    salary: 80000
    age: 30
    organizationId: 1
    departmentId: 1
  }) {
    id
    firstName
    lastName
  }
}
```

Шаг 4 — теперь запрос вернёт данные:

```
graphql
```

```
query {  
  organizations {  
    id  
    name  
    departments {  
      id  
      name  
    }  
  }  
}
```

Вопрос

О, всё заработало! Попрошу тебя добавить по 10 записей в базу данных в department, employee, organization.

Ответ

Написал bash-скрипт который через GraphQL mutations добавил в базу:

10 организаций: Google, Amazon, Microsoft, Apple, Meta, Netflix, Uber, Spotify, Twitter, Airbnb

10 департаментов: Engineering, Marketing, Finance, HR, Design, DevOps, QA, Sales, Legal, Analytics

10 сотрудников:

- Alice Johnson — Developer, зарплата 95000, возраст 28
- Bob Smith — Manager, зарплата 110000, возраст 35
- Carol Williams — Designer, зарплата 85000, возраст 26
- David Brown — DevOps, зарплата 100000, возраст 32
- Emma Jones — Analyst, зарплата 78000, возраст 29
- Frank Garcia — QA Engineer, зарплата 72000, возраст 31
- Grace Miller — HR Specialist, зарплата 68000, возраст 27
- Henry Davis — Sales Lead, зарплата 90000, возраст 40
- Iris Wilson — Marketing, зарплата 75000, возраст 33
- Jack Moore — Legal Counsel, зарплата 120000, возраст 45

Скрипт выполнен через curl с GraphQL mutations, после удалён. Данные в базе.

Следующие запросы для проверки:

graphql

```
# Все сотрудники с их департаментом и организацией
query {
  employees {
    firstName
    lastName
    position
    salary
    department { name }
    organization { name }
  }
}

# Фильтрация — сотрудники с зарплатой выше 90000
query {
  employeesWithFilter(filter: {
    salary: { operator: "gt", value: "90000" }
  }) {
    firstName
    lastName
    salary
    position
  }
}
```

Вопрос

Ты должен всю переписку сохранять подробно и грамотно — и мои вопросы и твои ответы. Не вижу что сохранены "Первые запросы — пишем вместе" с 5 запросами. Сделай и сохрани всё правильно то что в контексте нашей переписки работы.

Ответ

Перечитал файл — действительно, раздел с 5 запросами был сохранён в кратком виде, а три последних обмена отсутствовали. Переписал файл CHAT_HISTORY.md полностью с развёрнутым содержимым всех вопросов и ответов, включая все блоки кода и таблицы.

Вопрос

У меня в pom.xml две зависимости для GraphQL: spring-boot-starter-graphql и spring-graphql-test. Нужно ли добавить вот эти зависимости, что они дают и нужны ли мне? ``xml <dependency> <groupId>com.graphql-java</groupId> <artifactId>graphql-spring-boot-starter</artifactId> <version>5.0.2</version> </dependency> <dependency> <groupId>com.graphql-java</groupId> <artifactId>graphql-java-tools</artifactId> <version>5.2.4</version> </dependency> ``

Ответ

Не добавляй — они устарели и сломают проект.

Две эпохи GraphQL в Spring

Старый подход — `com.graphql-java` (2017–2021):

Community-библиотеки, появились когда Spring ещё не имел официальной поддержки GraphQL. Версия 5.x — это 2019 год. Использовали другую модель: интерфейсы-резолверы `GraphQLQueryResolver`, `GraphQLMutationResolver`.

```
java
// СТАРЫЙ стиль
@Component
public class DepartmentResolver implements GraphQLQueryResolver {
    public List<Department> departments() { ... }
}
```

Новый подход — `spring-boot-starter-graphql` (с 2022):

Официальная интеграция от Spring. Внутри уже содержит `graphql-java` (движок). Использует аннотации `@QueryMapping`, `@MutationMapping` — именно так в твоём проекте.

```
java
// НОВЫЙ стиль — именно это в твоём проекте
@Controller
public class DepartmentController {
    @QueryMapping
    public List<Department> departments() { ... }
    @QueryMapping
    public Department department(@Argument Integer id) { ... }
}
```

Что будет если добавить старые зависимости?

Конфликт — оба стартера попытаются зарегистрировать GraphQL сервлет на `/graphql`. Приложение не запустится или поведение станет непредсказуемым.

Зависимости в `pom.xml` — полная картина:

- `spring-boot-starter-graphql` — основа: GraphQL API, аннотации, схема
- `spring-graphql-test` — тестирование GraphQL запросов
- `spring-boot-starter-web` — HTTP сервер (GraphQL работает поверх HTTP)
- `spring-boot-starter-data-jpa` — JPA + Hibernate
- `postgresql` — драйвер БД
- `spring-webflux` (test scope) — нужен для тестирования через WebClient
- `lombok` — генерация геттеров/сеттеров

Всё что нужно для GraphQL — уже есть. Если видишь в интернете примеры с `GraphQLQueryResolver` — это устаревший код 2018-2020 годов, не применяй к своему проекту.

Сессия: 2026-05-10

Вопрос

GraphQL и REST – два разных подхода к разработке API для обмена данными через Интернет. REST дает возможность клиентским приложениям обмениваться данными с сервером с помощью команд HTTP – стандартного протокола связи в Интернете. А GraphQL – это язык запросов API, определяющий спецификации, по которым клиентское приложение должно запрашивать данные с удаленного сервера. Вы можете использовать GraphQL в вызовах API для определения запроса, не полагаясь на серверное приложение. GraphQL и REST – две мощные технологии, на которых основывается большинство наших современных приложений. GET, POST, PUT, DELETE, PATCH — объясни в чем разница?

Ответ

Когда мы проектируем или используем REST API, эти пять методов определяют тип действия, которое мы хотим совершить с ресурсом (например, пользователем, заказом или статьей).

1. GET — Получение данных

Используется исключительно для чтения информации. Он не должен изменять состояние сервера.

- Идемпотентность: Да (сколько бы раз вы ни вызывали GET, данные на сервере не изменятся)
- Безопасность: Да
- Пример: Получить список всех товаров в магазине

2. POST — Создание данных

Используется для создания нового ресурса. Данные передаются в теле запроса.

- Идемпотентность: Нет (если нажать «Отправить» дважды, в базе могут создаться две одинаковые записи, например, два заказа)
- Безопасность: Нет
- Пример: Регистрация нового пользователя или создание нового поста в блоге

3. PUT — Полное обновление

Используется для замены существующего ресурса целиком. Если ресурс не найден, PUT может создать его (зависит от реализации).

- Идемпотентность: Да (если вы отправите один и тот же объект 10 раз, результат будет одинаковым — объект просто перезапишется тем же самым)
- Важно: Нужно отправлять все поля объекта. Если вы отправите только имя, остальные поля (например, возраст или email) могут быть стёрты или заменены на значения по умолчанию

- Пример: Обновление профиля пользователя, где вы присылаете весь объект целиком

4. PATCH — Частичное обновление

Используется для изменения части ресурса.

- Идемпотентность: Обычно нет (но зависит от реализации)
- Пример: Вы хотите изменить только email пользователя, не затрагивая его имя, фамилию и дату рождения. Вы отправляете только это поле

5. DELETE — Удаление

Удаляет указанный ресурс.

- Идемпотентность: Да (первый вызов удалит ресурс, последующие не изменят состояние сервера, так как ресурса уже нет)
- Пример: Удаление фотографии из профиля

Сводная таблица

Метод	Действие	Идемпотентность	Тело запроса
GET	Чтение	Да	Нет
POST	Создание	Нет	Да
PUT	Полная замена	Да	Да
PATCH	Частичное изменение	Нет	Да
DELETE	Удаление	Да	Нет

Что такое идемпотентность?

Это важное понятие в архитектуре API. Метод называется идемпотентным, если повторный запрос к серверу с тем же набором данных приводит к тому же результату, что и первый успешный запрос. Это гарантирует стабильность системы при сетевых сбоях (например, если клиент не получил подтверждение и отправил запрос повторно).

Вопрос

Повторный запрос к серверу с тем же набором данных приводит к тому же результату — тогда как DELETE удаление того же результата не будет, один раз удалили и всё. Почему идемпотентным считается?

Ответ

Это отличный вопрос, который часто сбивает с толку. Разгадка кроется в том, что именно мы считаем «результатом».

В контексте идемпотентности результат — это состояние ресурсов на сервере, а не ответ сервера (код состояния HTTP).

Почему DELETE идиempotentен?

Представьте ситуацию:

Первый запрос DELETE /users/42:

- Сервер удаляет пользователя с ID 42
- Состояние сервера: пользователя 42 больше нет
- Ответ: 200 OK или 204 No Content

Второй запрос DELETE /users/42:

- Сервер видит, что пользователя 42 нет
- Состояние сервера: пользователя 42 по-прежнему нет — оно не изменилось по сравнению с состоянием после первого запроса
- Ответ: 404 Not Found

Несмотря на то что ответы разные (204 в первый раз и 404 во второй), конечное состояние базы данных одинаково — записи нет. Именно поэтому метод считается идиempotentным.

Сравнение с неидиempotentным POST

- Отправляете POST /orders → Сервер создаёт заказ №101
- Отправляете тот же POST /orders ещё раз → Сервер создаёт ещё один заказ — №102

После второго вызова состояние сервера изменилось: теперь там два заказа вместо одного. Это и есть отсутствие идиempotentности.

Коротко о главном

Идиempotentность — это про то, изменится ли мир (база данных) после второго, третьего или десятого идентичного запроса.

Для DELETE, PUT и GET мир меняется только один раз (или вообще не меняется), сколько бы раз вы ни дублировали запрос. Поэтому если из-за плохого интернета клиент отправил запрос на удаление дважды — это безопасно, вы просто удалите то, что уже удалено, и не повредите другие данные.

Вопрос

GraphQL как то связан с HTTP?

Ответ

Да, GraphQL напрямую связан с HTTP и в подавляющем большинстве случаев работает именно поверх этого протокола.

Хотя технически GraphQL является «протоколо-независимым» (его можно передавать через WebSocket, gRPC или даже через другие протоколы), на практике HTTP — это стандарт де-факто для передачи GraphQL-запросов.

Как GraphQL использует HTTP

В отличие от REST, где используются разные методы (GET, POST, PUT, DELETE) и множество разных URL, GraphQL упрощает схему:

- Один Endpoint: Обычно это один URL (например, /graphql)
- Один метод — POST: Почти все запросы в GraphQL отправляются методом POST. Даже если вы просто хотите «прочитать» данные (что в REST делается через GET), вы отправляете JSON-тело с описанием запроса внутри POST-запроса

> Примечание: Иногда для простых Query-запросов используют GET с параметрами в URL, но это встречается реже из-за ограничений по длине строки.

Структура запроса

В REST HTTP-метод определяет действие. В GraphQL действие (чтение, запись или подписка) определяется внутри тела запроса, а не средствами HTTP.

- REST: GET /users/1
- GraphQL: POST /graphql с телом `query { user(id: 1) { name } }`

Обработка статус-кодов HTTP

В REST мы сильно полагаемся на статус-коды: 201 (создано), 404 (не найдено), 500 (ошибка сервера).

В GraphQL сервер почти всегда возвращает статус-код 200 OK, даже если в самом запросе произошла ошибка. Информация об ошибках передается внутри JSON-ответа в специальном массиве "errors".

Главные отличия в контексте HTTP

Характеристика	REST	GraphQL
Количество URL	Много (/users, /posts, /orders)	Один (/graphql)
Методы HTTP	Использует весь спектр (GET, POST и т.д.)	В основном только POST
Кэширование	На уровне HTTP (браузером, CDN)	На уровне клиента (Apollo, Relay)
Статус-коды	Отражают результат операции	Почти всегда 200 OK

Итог: GraphQL не заменяет HTTP, он использует его как «транспорт». Если REST относится к HTTP как к набору команд, то GraphQL относится к HTTP как к простому туннелю, по которому передаётся его собственный сложный язык запросов.

Вопрос

В GraphQL что такое Graph и что такое QL?

Ответ

Название GraphQL расшифровывается как Graph Query Language (язык запросов к графам).

1. Что такое Graph (Граф)?

В контексте GraphQL ваши данные представляются не в виде плоских таблиц (как в Excel), а в виде графа — сети связанных между собой объектов.

- Узлы (Nodes): Это объекты (например, «Пользователь», «Пост», «Комментарий»)
- Рёбра (Edges): Это связи между ними (например, пользователь «написал» пост, пост «содержит» комментарии)

В REST API данные часто разрознены: чтобы получить профиль пользователя и его последние 10 постов, нужно сделать два разных запроса. В GraphQL вы просто «идёте» по графу от одного узла к другому за один запрос.

Аналогия: Представьте социальную сеть. Вы — точка в центре. От вас идут линии к вашим друзьям, от друзей — к их интересам. Это и есть граф. GraphQL позволяет сказать: «Начни с меня, перейди к моим друзьям, а от них — к их фотографиям».

2. Что такое QL (Query Language)?

Query Language — это «язык запросов». Точно так же, как SQL используется для общения с базами данных, GraphQL используется для общения с вашим API.

Ключевая особенность: клиент сам выбирает, что ему нужно.

```
graphql
query {
  user(id: "123") {
    name          # Хочу только имя
    friends {     # Имена его друзей
      name
    }
  }
}
```

Сервер посмотрит на этот «запрос» и вернёт JSON ровно такой же структуры. Никаких лишних данных.

Почему это работает вместе?

Смысл GraphQL в том, чтобы дать мощный язык (QL) для навигации по огромной сети связанных данных (Graph). Вы не думаете о «эндпоинтах» или «таблицах». Вы просто берёте «карту» ваших данных и с помощью специального синтаксиса указываете путь, по которому нужно пройти, чтобы собрать нужную информацию за один заход.

Вопрос

В чём же преимущество GraphQL если база данных Postgres и надо достать пользователя, посты и комментарии — всё равно же будет три запроса разных к базе данных. Зачем нам тогда GraphQL если можно применять REST?

Ответ

Это отличный вопрос, который бьёт прямо в «больную мозоль» GraphQL — проблему N+1.

Вы абсолютно правы: если просто написать код «в лоб», то для получения пользователя, его 10 постов и комментариев к ним, GraphQL-сервер может сделать 1 запрос к пользователю, 1 к постам и 10 к комментариям (всего 12!). Это будет даже хуже, чем в REST.

Но преимущество GraphQL раскрывается на двух уровнях: клиентском и инженерном.

1. Со стороны фронтенда (Клиент)

- В REST: Нужно написать код для трёх вызовов по сети. Нужно обработать три ожидания (loading), три возможные ошибки и склеить эти данные на клиенте. Это тратит заряд батареи телефона и время пользователя (лишние «рукопожатия» HTTP).
- В GraphQL: Делается один сетевой запрос. Весь хаос со сбором данных переносится на сервер, где связь с базой данных в сотни раз быстрее, чем связь телефона с вышкой 4G.

2. Со стороны бэкенда — решение проблемы запросов (DataLoader)

Чтобы GraphQL не «засыпал» Postgres кучей мелких запросов, используются специальные инструменты — DataLoader.

Вместо того чтобы выполнять запросы сразу, DataLoader «собирает» все ID, которые нужно достать, и объединяет их в один запрос:

```
sql
```

```
SELECT * FROM comments WHERE post_id IN (1, 2, 3...)
```

В итоге к базе данных всё равно уходит минимум запросов (часто те же 2-3), но клиент получает всё «одним пакетом».

3. Главные преимущества (почему не REST?)

Ситуация	REST	GraphQL
Overfetching	Сервер отдаёт все поля профиля (даже если нужно только имя). Лишний трафик.	Клиент запрашивает только name. Экономия трафика.

Ситуация	REST	GraphQL
Underfetching	Одного запроса мало, нужна цепочка user → posts → comments.	Один запрос забирает всё дерево данных сразу.
Версионность	При изменении структуры нужна /v2/, /v3/.	API эволюционирует плавно. Просто добавляются новые поля.
Типизация	Нужна документация (Swagger), которая может врать.	Строгая схема (Schema). Клиент всегда знает, какие данные можно получить.

Резюме

GraphQL не заменяет SQL и не делает магию внутри базы данных. Его главная цель — отвязать фронтенд от структуры бэкенда.

- Если у вас простое CRUD-приложение, где вы всегда выводите одну таблицу — REST проще и лучше.
- Если у вас сложный интерфейс (как лента Facebook или Dashboard Jira), где в разных местах нужны разные кусочки связанных данных — GraphQL экономит месяцы разработки и делает работу приложения быстрее для конечного пользователя.